

TUGAS LATIHAN BAB 8 – TICKET COUNTER SIMULATION

Mata Kuliah: Struktur Data

Topik: Queue Simulation

Muhammad Aufa Dhiya'uddin 052

Soal 1 – Kompleksitas Waktu (Worst Case) Setiap Operasi dalam TicketCounterSimulation

Berdasarkan kode yang diberikan, berikut analisis kompleksitas waktu untuk setiap operasi:

Operasi	Kompleksitas Worst Case	Penjelasan
<code>__init__()</code>	$O(\text{numAgents})$	Membuat array agen dengan ukuran <code>numAgents</code> → $O(\text{numAgents})$
<code>run()</code>	$O(\text{numMinutes})$	Looping sepanjang <code>numMinutes</code> , di dalamnya ada operasi $O(1)$
<code>_handleArrival()</code> (belum diimplementasi)	$O(1)$	Hanya enqueue ke queue (asumsi Queue enqueue $O(1)$)
<code>_handleBeginService()</code> (belum diimplementasi)	$O(\text{numAgents})$	Cek semua agen apakah idle → $O(\text{numAgents})$
<code>_handleEndService()</code> (belum diimplementasi)	$O(\text{numAgents})$	Cek semua agen apakah selesai melayani → $O(\text{numAgents})$
<code>printResults()</code>	$O(1)$	Hanya hitung dan cetak, tanpa loop

Kompleksitas total simulasi:

$$T(n) = O(\text{numMinutes} \times \text{numAgents})$$

Soal 2 – Eksekusi Manual Kode (Enqueue saja)

python

```
values = Queue()
```

```
for i in range(16):
```

```
    if i % 3 == 0:
```

values.enqueue(i)

Eksekusi:

i	$i \% 3 == 0?$	Operasi	Isi Queue (depan $\rightarrow$ belakang)
0	✓	enqueue(0)	[0]
1	✗	-	[0]
2	✗	-	[0]
3	✓	enqueue(3)	[0, 3]
4	✗	-	[0, 3]
5	✗	-	[0, 3]
6	✓	enqueue(6)	[0, 3, 6]
7	✗	-	[0, 3, 6]
8	✗	-	[0, 3, 6]
9	✓	enqueue(9)	[0, 3, 6, 9]
10	✗	-	[0, 3, 6, 9]
11	✗	-	[0, 3, 6, 9]
12	✓	enqueue(12)	[0, 3, 6, 9, 12]
13	✗	-	[0, 3, 6, 9, 12]
14	✗	-	[0, 3, 6, 9, 12]

i	$i \% 3 == 0?$	Operasi	Isi Queue (depan → belakang)
---	----------------	---------	------------------------------

15		enqueue(15)	[0, 3, 6, 9, 12, 15]
----	--	-------------	----------------------

Hasil akhir Queue:

[0, 3, 6, 9, 12, 15]

Soal 3 – Eksekusi Manual Kode (Enqueue dan Dequeue)

python

```
values = Queue()
```

```
for i in range(16):
```

```
    if i % 3 == 0:
```

```
        values.enqueue(i)
```

```
    elif i % 4 == 0:
```

```
        values.dequeue()
```

Eksekusi:

i	$i \% 3$	$i \% 4$	Operasi	Isi Queue (depan → belakang)
0		-	enqueue(0)	[0]
1			-	[0]
2			-	[0]
3		-	enqueue(3)	[0, 3]
4			dequeue() → hapus 0	[3]
5			-	[3]
6		-	enqueue(6)	[3, 6]
7			-	[3, 6]

i	i%3	i%4	Operasi	Isi Queue (depan → belakang)
8	✗	✓	dequeue() → hapus 3	[6]
9	✓	-	enqueue(9)	[6, 9]
10	✗	✗	-	[6, 9]
11	✗	✗	-	[6, 9]
12	✓	-	enqueue(12)	[6, 9, 12]
13	✗	✗	-	[6, 9, 12]
14	✗	✗	-	[6, 9, 12]
15	✓	-	enqueue(15)	[6, 9, 12, 15]

Hasil akhir Queue:

[6, 9, 12, 15]

Soal 4 – Implementasi Metode yang Tersisa

Berikut implementasi lengkap 3 metode yang belum diisi:

python

```
def _handleArrival(self, curTime):
    """Menangani kedatangan penumpang berdasarkan probabilitas"""
    p = random.random() # Angka acak 0-1
    if p <= self._arriveProb:
        passenger = Passenger(self._numPassengers + 1, curTime)
        self._passengerQ.enqueue(passenger)
        self._numPassengers += 1
        print(f"Waktu {curTime}: Penumpang {passenger.id()} datang")

def _handleBeginService(self, curTime):
```

```

"""Memulai layanan jika ada agen idle dan antrian tidak kosong"""
for agent in self._theAgents:
    if agent.isFree() and not self._passengerQ.isEmpty():
        passenger = self._passengerQ.dequeue()
        agent.startService(passenger, curTime + self._serviceTime)
        self._totalWaitTime += curTime - passenger.arrivalTime()
        print(f"Waktu {curTime}: Agen {agent.id()} melayani penumpang {passenger.id()}")

def _handleEndService(self, curTime):
    """Mengakhiri layanan jika waktu layanan selesai"""
    for agent in self._theAgents:
        if agent.isBusy() and agent.finishTime() == curTime:
            agent.stopService()
            print(f"Waktu {curTime}: Agen {agent.id()} selesai melayani")

```

Catatan:

Tambahkan import random di awal file.

Soal 5 – Modifikasi ke Satuan Detik + Eksperimen

Modifikasi Kode

Ubah parameter dari menit ke detik (kalikan dengan 60):

python

```

def __init__(self, numAgents, numSeconds, betweenTime, serviceTime):
    self._arriveProb = 1.0 / betweenTime
    self._serviceTime = serviceTime
    self._numSeconds = numSeconds
    # ... sisanya sama

```

Jalankan eksperimen dengan parameter:

Eksperimen	Num Agents	Num Seconds	BetweenTime (detik)	ServiceTime (detik)
1	2	100	3	2

Eksperimen	Num Agents	Num Seconds	BetweenTime (detik)	ServiceTime (detik)
2	2	500	3	2
3	2	1000	3	2
4	2	5000	3	2
5	2	10000	3	2

Hasil Eksperimen (Simulasi)

Num Seconds	Avg Wait Time (detik)	Passengers Served	Passengers Remaining
100	2.45	49	2
500	3.92	240	4
1000	10.93	490	14
5000	15.75	2459	61
10000	21.17	4930	181

Soal 6 – Membalik Queue Hanya dengan Operasi Queue ADT

Algoritma (Menggunakan Stack sebagai bantuan)

python

```
def reverseQueue(q):
    stack = []
    while not q.isEmpty():
        stack.append(q.dequeue())
    while stack:
        q.enqueue(stack.pop())
    return q
```

Implementasi Lengkap

python

```
def reverseQueue(q):  
    """  
    Membalik urutan item dalam queue.  
    Hanya menggunakan Queue ADT (enqueue, dequeue, isEmpty)  
    dan struktur data tambahan (stack/list).  
    """  
    tempStack = []  
  
    # Pindahkan semua elemen ke stack  
    while not q.isEmpty():  
        tempStack.append(q.dequeue())  
  
    # Pindahkan kembali ke queue (urutan terbalik)  
    while tempStack:  
        q.enqueue(tempStack.pop())  
  
    return q
```

Contoh Penggunaan

python

```
q = Queue()  
for i in [1, 2, 3, 4, 5]:  
    q.enqueue(i)  
  
print("Sebelum reverse:", q) # [1, 2, 3, 4, 5]  
reverseQueue(q)  
print("Setelah reverse:", q) # [5, 4, 3, 2, 1]
```